electronics

MDPI

Article

EcoInfer: Optimizing Energy Efficiency with Latency Guarantees Through Iteration-Level GPU Frequency Control in LLM Serving

Qingyuan Hu and Jian Li *

School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China; huqingyuan@sjtu.edu.cn
* Correspondence: li-jian@sjtu.edu.cn

Abstract

Large language model (LLM) serving has emerged as a major source of energy consumption in modern AI infrastructure. In current deployments, graphics processing units (GPUs) are typically operated at default high-frequency settings to maximize performance. However, under practical service-level objectives (SLOs), peak performance is often unnecessary, especially during the memory-bound decode stage, resulting in substantial power redundancy and avoidable energy waste. Existing studies that apply GPU dynamic voltage and frequency scaling (DVFS) to improve the energy efficiency of LLM serving have shown promising results. However, they generally rely on coarse-grained control, accurate output length prediction, or request-level resource management, which limits their effectiveness under highly dynamic workloads and strict SLO constraints. We present EcoInfer, a fine-grained DVFS framework for energy-efficient LLM serving. EcoInfer performs iteration-level, workload-aware GPU frequency control that adapts to the current inference phase and system state while preserving latency guarantees. It comprises three tightly integrated modules: a machine-learning-based frequency–latency predictor that estimates iteration latency across candidate GPU frequencies using lightweight iteration-level features; an SLO-aware frequency controller that selects the minimum feasible frequency within a sweet-spot-guided candidate range; and a low-overhead runtime optimization layer that combines adaptive decision caching with asynchronous execution to reduce and hide the overhead of online control. Implemented on top of vLLM, EcoInfer achieves up to 25.4% energy savings and 21.5% average energy savings and improves energy efficiency by 1.28× on average in terms of Tokens/J while maintaining a nearly unchanged SLO attainment rate compared with the default vLLM baseline.

Keywords: LLM serving systems; GPU DVFS; energy-efficient LLM serving; SLO-aware frequency control; iteration-level latency prediction

Check for updates

Academic Editor: David Defour

Received: 2 April 2026
Revised: 4 May 2026
Accepted: 5 May 2026
Published: 16 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Large language model (LLM) serving has become a critical workload in modern data centers, with the ever-growing demand for AI-powered applications driving substantial energy consumption at scale [1,2]. In production deployments, graphics processing units (GPUs) are typically operated at their default maximum frequency to deliver peak throughput. However, real-world LLM serving is predominantly governed by service-level objectives (SLOs)—such as time to first token (TTFT) and time between tokens (TBT)—rather than raw throughput maximization [3–6]. Moreover, request arrival rates fluctuate over time, meaning that the system is not always under peak load. As a result, running GPUs at the highest frequency at all times is often unnecessary to meet SLO targets, leading to

significant power redundancy and avoidable energy waste. GPU dynamic voltage and frequency scaling (DVFS) [7], which adjusts the core clock frequency at runtime, provides a promising mechanism to reclaim this wasted energy. Simply setting a GPU power cap to reduce GPU frequency can lower dynamic power consumption, but it may incur severe tail latency and thus violate inference SLO constraints. Balancing energy savings against latency degradation is therefore the central challenge in applying DVFS to LLM serving.

Recent studies [2,8–12] have begun to explore DVFS-based energy optimization for LLM serving. DynamoLLM [11] routes incoming requests to GPU pools configured at different static frequencies based on predicted output token lengths, achieving energy savings at the cluster level. ThrottLL'eM [12] introduces a KV cache prediction mechanism to determine GPU frequency before request execution. Despite their promising results, both approaches share fundamental limitations. First, they operate at coarse granularity and cannot adapt frequency to the rapidly changing computational characteristics within online serving iterations. Second, they rely on accurate predictions of output sequence lengths, which are inherently uncertain in production workloads and introduce fragility.

Our investigation reveals three key observations that motivate finer-grained GPU frequency control for LLM serving. **First**, lowering GPU frequency does not always reduce total energy consumption. Since energy is determined by both power and execution time, $E(f) = P(f) \times T(f)$, the energy–frequency relationship typically exhibits a U-shaped trend: reducing frequency is beneficial when power savings dominate but becomes counterproductive once latency increases too much. As a result, each workload has an *energy sweet spot*, and this sweet spot varies with workload characteristics. Therefore, effective DVFS must select frequency dynamically rather than rely on a fixed setting. **Second**, the prefill and decode phases respond very differently to GPU frequency scaling. Prefill is largely compute-bound and thus highly sensitive to frequency reduction, whereas decode is more memory-bound and often tolerates lower frequencies with only modest latency increase. This asymmetry makes a uniform-frequency strategy inherently inefficient: a low frequency harms TTFT, while a high frequency wastes energy during decode. Hence, DVFS for LLM serving must operate at fine granularity and remain sensitive to the different latency characteristics contributed by prefill and decode work. **Third**, recent trends in LLMs further enlarge the opportunity for DVFS-based energy optimization. Architectural and workload shifts such as Mixture-of-Experts (MoE) models, longer decode-heavy reasoning, and the growing imbalance between GPU compute throughput and memory bandwidth are pushing inference toward a more memory-bound regime. In such settings, frequency reduction tends to incur smaller performance penalties while still delivering meaningful power savings, making fine-grained DVFS increasingly attractive.

Based on these observations, we present **EcoInfer**, a fine-grained DVFS framework for energy-efficient LLM serving that performs iteration-level, SLO-aware GPU frequency control. EcoInfer comprises three tightly integrated components. First, an *Iteration-Level Frequency–Latency Predictor* estimates the latency of the current serving iteration under candidate GPU frequencies using iteration features available at runtime, without relying on output length prediction. Second, an *SLO-Aware Frequency Controller* uses these predicted latencies together with the scheduler-provided iteration budget to select the minimum feasible frequency within a sweet-spot-guided candidate range, thereby achieving low-energy operation while preserving latency guarantees. Third, a *low-overhead runtime optimization layer* makes iteration-level control practical through a bucketized frequency-decision cache and asynchronous pipelining with decoupled DVFS actuation, which together reduce and hide the overhead of online decision making and frequency enforcement.

The main contributions of our work are as follows:

- A systematic analysis and experimental characterization of GPU DVFS behavior in LLM serving, revealing that (i) an *energy sweet spot* exists whose location varies dynamically with workload conditions, (ii) the prefill and decode phases exhibit fundamentally different frequency sensitivities that preclude uniform-frequency strategies, and (iii) the ongoing shift toward memory-bound inference—driven by MoE architectures, chain-of-thought reasoning, and hardware evolution—is systematically expanding the DVFS energy-saving opportunity.
- The proposal of EcoInfer, an iteration-level DVFS framework that integrates a lightweight frequency–latency predictor, an SLO-aware frequency controller, and low-overhead runtime optimizations to achieve fine-grained, energy-efficient GPU frequency control for LLM serving under latency constraints.
- The implementation and evaluation of EcoInfer atop vLLM, demonstrating up to 25.4% energy savings (21.5% on average) and a 1.28× improvement in energy efficiency measured in tokens per joule (TPJ) while maintaining a nearly unchanged SLO attainment rate compared with default-frequency operation.

The remainder of this paper is organized as follows. Section 2 introduces background on LLM serving, GPU DVFS, and related work. Section 3 presents our motivating observations and formulates the challenges. Section 4 details the design and implementation of EcoInfer. Section 5 reports the experimental evaluation. Section 6 discusses the implications, limitations, and future directions of EcoInfer. Section 7 concludes this paper.

2. Background and Related Work

This section introduces the necessary background on LLM serving and GPU DVFS, and reviews related work in inference efficiency and energy-efficient LLM serving.

Throughout this paper, we use the following terminology consistently. A phase refers to one of the two stages of LLM autoregressive generation, namely, prefill or decode. A batch is the set of requests packed together by the serving scheduler at a given step. An iteration is a single forward pass of the model that executes one batch. Under continuous batching, a single iteration may contain prefill and decode work for different requests simultaneously (a mixed batch).

2.1. LLM Serving System

2.1.1. Two-Phase Inference Pipeline

Modern LLM inference follows an autoregressive generation paradigm consisting of two phases with distinct computational characteristics [13]. In the prefill phase, the model processes the entire input prompt in parallel, computes the key–value (KV) pairs for all prompt tokens, and stores them for subsequent reuse. This phase is dominated by large matrix operations with $O(n^2 d)$ complexity [14], where n denotes the sequence length and d the hidden dimension, and is generally compute-bound. In the decode phase, the model generates output tokens iteratively, one token per step. Each step loads the model parameters together with the accumulated KV cache from high-bandwidth memory (HBM) to produce the next token, resulting in $O(nd)$ complexity. In contrast to prefill, decode is typically memory-bandwidth-bound [3,15,16].

2.1.2. Service Level Objectives

LLM serving quality is governed by SLOs along two latency dimensions: TTFT and TBT [3–5]. TTFT is the delay from request submission to the first generated token, determined primarily by the waiting time before execution and the prefill-phase execution latency. TBT is the delay between consecutive tokens during decode. The SLO attainment

rate—the fraction of requests meeting both TTFT and TBT targets—is the primary quality-of-service metric in production deployments [6].

2.2. GPU Dynamic Voltage and Frequency Scaling

GPU DVFS adjusts the GPU core clock frequency at runtime to modulate the power–performance trade-off [7,17,18]. In CMOS circuits, dynamic power scales as $V^2 f$ [19]. Since the supply voltage V scales approximately linearly with frequency f, the total GPU power can be modeled as:

$$P(f) = a \cdot f^2 + b \cdot f + c, \tag{1}$$

where c accounts for the static leakage power and frequency-independent baseline power; b captures the linear dynamic power scaling, particularly dominant when voltage scaling hits its lower bound; and a reflects the quadratic dynamic power scaling induced by joint voltage-frequency adjustments [20,21].

As illustrated in Figure 1, GPU power increases monotonically with operating frequency, providing a direct and effective control knob for runtime energy management. NVIDIA data center GPUs expose a wide DVFS range—for example, the A100 supports core frequencies from 210 MHz to 1410 MHz—and frequency changes can be applied programmatically via the pyNVML library [22]. DVFS updates can be issued while GPU kernels are executing. The clock transition occurs over a finite hardware transition interval and manifests primarily as frequency-dependent execution-time variation rather than requiring the GPU to become idle or causing a device-wide execution stall [18,23].

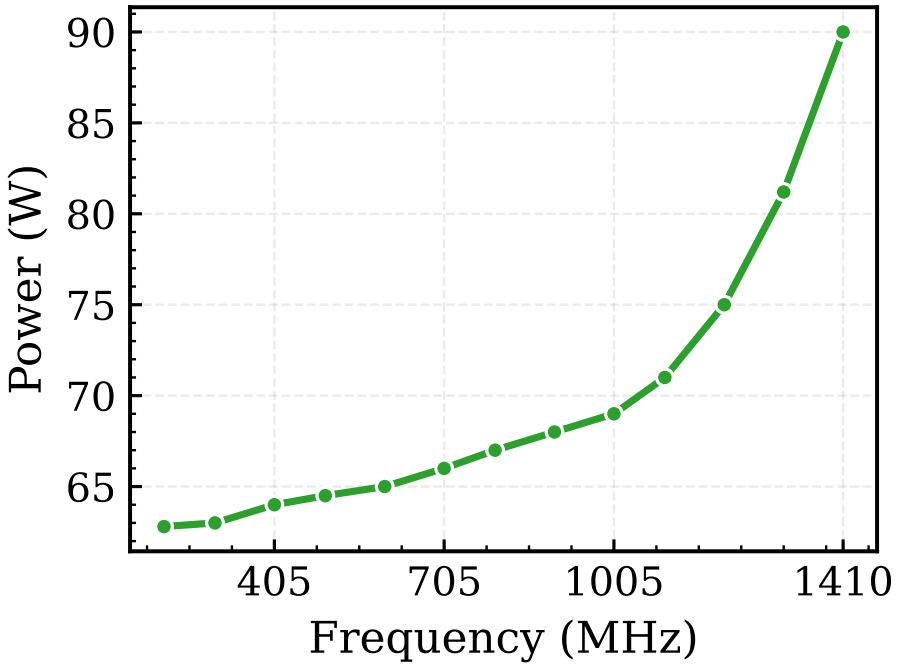


Figure 1. Relationship between GPU core frequency and power consumption under DVFS. GPU power increases monotonically with frequency and can be well approximated by a quadratic function, consistent with Equation (1).

A key empirical observation is that GPU power consumption is predominantly governed by operating frequency rather than computational load intensity: under a fixed frequency, the power standard deviation across different batch sizes is less than 3.5% [12]. This high predictability makes DVFS a viable and quantifiable control knob for fine-grained energy management in LLM serving.

2.3. Related Work

2.3.1. LLM Serving Efficiency Optimization

A rich body of work has targeted LLM serving efficiency from multiple directions. In scheduling, Orca [24] introduced iteration-level scheduling with continuous batching; Sarathi-Serve [3] proposed chunked prefill for latency-throughput balancing; and Llumnix [25] enabled runtime request migration across model instances for load balancing. In KV cache management, PagedAttention [15] (implemented in vLLM) applied virtual-memory-inspired paged allocation to eliminate memory fragmentation, while MoonCake [26] proposed a global KV cache pool for efficient prefix reuse. In kernel optimization, FlashAttention [27] reduced attention HBM accesses from $O(N^2)$ to $O(N)$ via IO-aware tiling. In distributed inference, DistServe [4] and Splitwise [28] introduced prefill–decode disaggregation, and MegaScale-Infer [29] further separated attention and feed-forward network (FFN) layers onto heterogeneous hardware. These works mainly focused on improving the inference efficiency of LLM serving systems. While they sought to improve throughput and latency from multiple perspectives, they did not address another critical factor that affects both operational cost and carbon footprint: substantial energy consumption.

2.3.2. Energy-Efficient LLM Serving

Research on energy-efficient LLM inference remains nascent, and recent studies [2,8–12] have begun exploring energy-saving and power-management approaches for LLM inference.

Patel et al. [10] presented the first systematic characterization of power consumption patterns for LLM workloads in cloud environments. Their study analyzed power differences between the training and inference phases and identified opportunities for cluster-level power management, demonstrating that inference workloads offer substantial headroom for power oversubscription. However, the work remained at the characterization level and did not provide a deployable energy optimization framework.

DynamoLLM [11] proposes a dynamic energy optimization framework targeting large-scale inference clusters. It employs a hierarchical design that routes incoming requests to dedicated GPU pools configured with different model parallelism degrees and GPU frequencies, selecting configurations based on predicted output token lengths and adjusting frequency at a coarse granularity according to length characteristics. Despite its promising cluster-level savings, DynamoLLM has several limitations. First, its frequency adjustment lacks fine-grained awareness of the computational characteristics within individual requests (e.g., the prefill–decode phase distinction). Second, it assumes relatively stable workload patterns, making it vulnerable to bursty traffic or highly heterogeneous requests, which increases SLO violation risk. Third, it relies heavily on accurate predictions of output token lengths—a quantity with high uncertainty in production environments—which leads to suboptimal resource allocation and undermines system reliability.

ThrottLL'eM [12] introduces a KV-cache-based prediction mechanism to estimate future computational demand and determine GPU frequency at request scheduling time. This design is effective in certain scenarios but still has significant limitations. First, although its SLO enforcement covers both end-to-end P99 latency and average TBT, it leaves TTFT outside its control loop. Second, like DynamoLLM, it relies on accurate output-length prediction, which may limit its accuracy under dynamic, uncertain workloads. Third, ThrottLL'eM still operates at request-level granularity: frequency is fixed at scheduling time and cannot adapt to intra-request variations such as prefill–decode phase shifts.

In summary, these pioneering efforts demonstrate early opportunities for DVFS-based energy savings in LLM serving, yet they share common limitations: coarse (request- or pool-level) control granularity, dependence on output-length prediction, and absence of

phase-aware frequency differentiation. Table 1 consolidates this comparison along five dimensions that most strongly affect energy efficiency under SLO constraints: control granularity, predictive signal, SLO awareness, dependence on output-length prediction, and intra-request phase-aware adaptation.

Table 1. Conceptual comparison of representative DVFS-based energy-efficient LLM serving systems. EcoInfer differs from prior systems by combining iteration-level SLO-aware frequency control with no dependence on output-length prediction and phase-aware adaptation within a single request.

System	Control Granularity	Predictive Signal	SLO Awareness	Out.-Len. Pred.	Phase-Aware
DynamoLLM [11]	Pool (5 s epoch)	Output length pred. + load pred.	TTFT + TBT (per pool)	Required	No
ThrottLL'eM [12]	Request	Future (KV, batch) trajectory via output-len. pred.	E2E P99 + TBT	Required	No
EcoInfer (ours)	Iteration	Exact current-iter. batch composition	TTFT + TBT (per iter.)	Not required	Yes

3. Motivation and Challenges

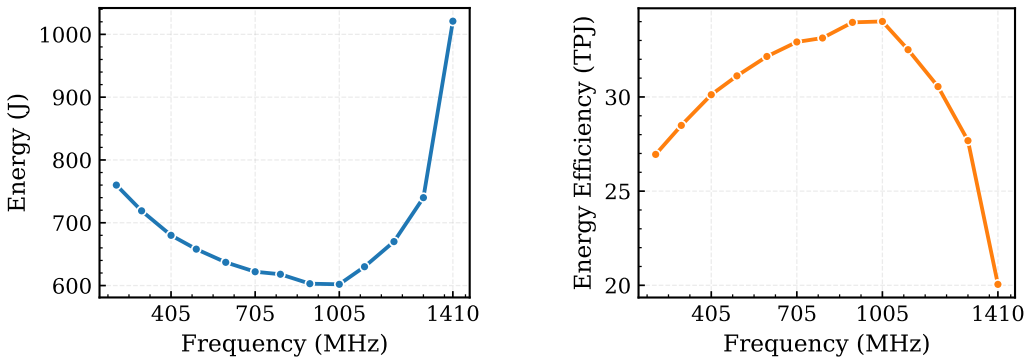
This section presents three experimental observations that collectively reveal both the opportunity and the non-triviality of applying fine-grained GPU DVFS to energy-efficient LLM serving under latency constraints, followed by the key technical challenges that arise from these observations. Unless otherwise stated, all profiling experiments are conducted on NVIDIA A100-80G GPUs serving Qwen3-32B [30] with the ShareGPT [31] workload trace.

3.1. Observation 1: The Energy Sweet Spot in LLM Serving

As discussed in Section 2.2, GPU power under DVFS follows a quadratic relationship with frequency, $P(f) = af^2 + bf + c$. Inference latency, however, responds to frequency scaling sublinearly rather than proportionally. Consequently, the total energy consumed by an inference task,

$$E(f) = P(f) \times T(f), \tag{2}$$

forms a characteristic U-shaped curve with respect to GPU frequency, as shown in Figure 2. Strictly speaking, iteration energy is the integral $E = \int_0^{T(f)} P(t, f)\, \mathrm{d}t$. The product form in Equation (2) is an average-power approximation that is accurate at a fixed GPU frequency.



(**a**) Energy consumption. (**b**) Energy efficiency.

Figure 2. Energy consumption and efficiency together reveal an *energy sweet spot* in LLM inference under GPU frequency scaling. (**a**) Energy consumption exhibits a U-shaped trend across GPU frequencies. (**b**) Energy efficiency, measured in TPJ, shows an inverted-U trend, indicating that the most energy-efficient operating point lies at an intermediate frequency rather than at either extreme. Results are obtained by serving Qwen3-8B [30] on an A100-80G using vLLM [15].

Figure 2 reveals a clear *energy sweet spot* in LLM inference under GPU frequency scaling. This operating point corresponds to the global minimum of total energy consumption $E(f)$. Importantly, it lies at neither extreme: running at the maximum frequency wastes power, whereas pushing frequency too low increases execution time enough to offset the power savings. Instead, the sweet spot emerges from the nonlinear trade-off between power reduction and latency growth. Above this point, lowering frequency reduces energy because power drops faster than execution time increases. Below it, execution time rises sharply enough that total energy increases again.

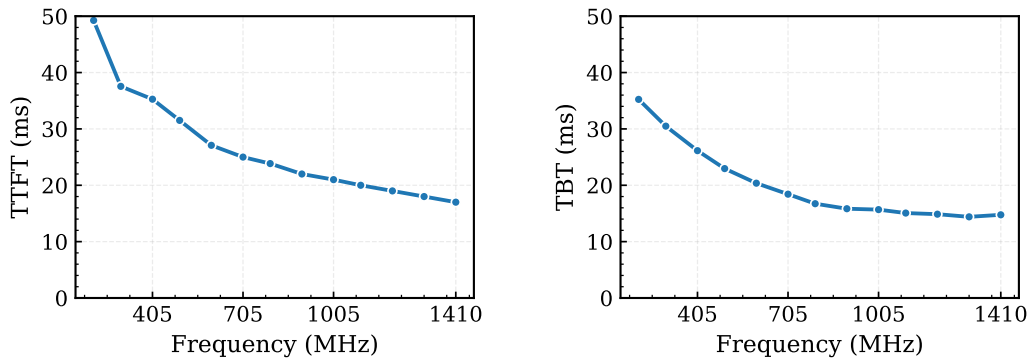
Implication.

LLM inference exhibits a workload-dependent *energy sweet spot* under GPU DVFS. This property creates a concrete opportunity for energy optimization in LLM serving: by steering GPU frequency toward the sweet spot, rather than always operating at the highest or lowest frequency, the system can reduce energy more effectively.

3.2. Observation 2: Divergent Frequency Sensitivity Across Inference Phases

The two-phase structure of LLM inference (Section 2.1.1) yields markedly different responses to frequency scaling. To quantify this difference, we profile TTFT and TBT across a range of GPU frequencies.

As shown in Figure 3, the prefill phase is highly sensitive to frequency reduction: every 10% decrease in GPU frequency increases TTFT by 8–12%, reflecting its compute-bound nature. In contrast, the decode phase exhibits much lower sensitivity: a 50% frequency reduction increases per-token latency by less than 20%, consistent with its memory-bandwidth-bound profile. This disparity arises because decode iterations are bottlenecked by HBM bandwidth rather than arithmetic throughput. Therefore, reducing core frequency has a limited impact on the critical path.



(**a**) Prefill: TTFT vs. GPU frequency. (**b**) Decode: TBT vs. GPU frequency.

Figure 3. Latency sensitivity to GPU frequency scaling. (**a**) Prefill-phase TTFT increases sharply with frequency reduction. (**b**) Decode-phase TBT remains relatively stable even under aggressive frequency scaling, particularly when the GPU frequency is above 705 MHz.

This asymmetry becomes even more consequential in practical online serving, where continuous batching and chunked prefill cause the workload to change rapidly across consecutive iterations, with batch composition shifting among prefill-dominated, decode-dominated, and mixed batches of different sizes and token counts. It therefore exposes a fundamental limitation of uniform-frequency strategies—whether static or request-level. Applying a single frequency to both phases forces an undesirable trade-off: a frequency low enough to yield meaningful decode-phase savings would cause severe TTFT degradation and SLO violations, whereas a frequency high enough to protect prefill-phase SLOs leaves the dominant decode-phase energy largely unoptimized.

Implication.

Effective DVFS control for LLM serving must operate at iteration-level granularity with phase-aware frequency differentiation, selecting distinct frequencies for iterations based on their real-time computational characteristics.

3.3. Observation 3: Intensified Memory Wall Amplifies Energy-Saving Potential

As LLM architectures, inference paradigms, and infrastructure continue to evolve rapidly, three concurrent trends are driving LLM inference toward an increasingly severe memory wall, thereby increasing the potential for DVFS-based energy savings.

Trend 1: Increasingly sparse model architectures.

The shift from dense to MoE models [32,33] tends to lower arithmetic intensity, making inference more memory-bound. Compared with dense models, MoE models activate only a small subset of experts for each token, as illustrated in Figure 4, which keeps per-token computation relatively low. However, under batched execution, different tokens are often routed to different experts. As a result, the system must fetch the union of all experts activated by the batch rather than repeatedly reusing a single shared FFN as in dense models. This increases memory access volume (MAV) and often weakens weight reuse. Consequently, computation does not grow proportionally with memory traffic, and the arithmetic intensity, AI = FLOPs/MAV, decreases, pushing MoE inference further toward the memory-bound regime.

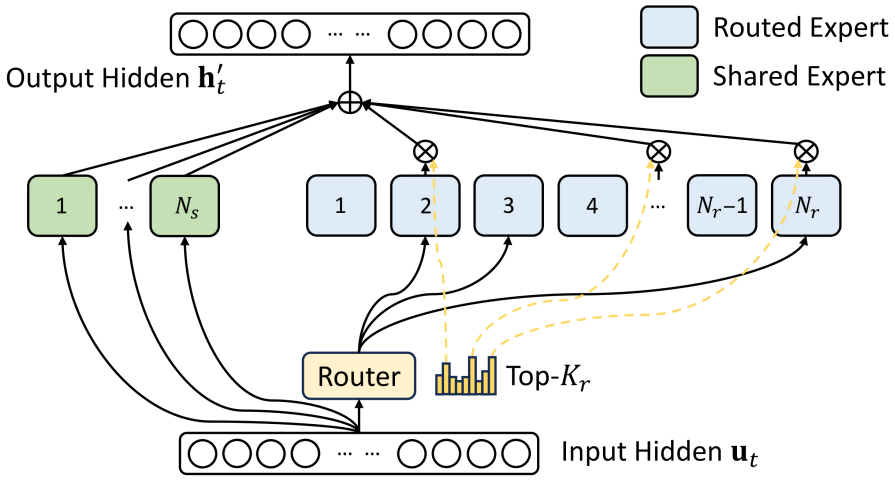


Figure 4. Overview of the MoE architecture, excerpted from the DeepSeek-V3 Technical Report [34]. For each token, the router selects only a small subset of experts, while different tokens in the same batch may activate different experts.

To quantify this effect, we define MAV as the total weight data read from HBM during inference. Let L denote the number of layers, d the hidden dimension, n the batch token count, K the number of activated experts per layer, and d_{e}, d_{r} the expert and router dimensions, respectively:

$$\mathrm{MAV}_{\mathrm{dense}} = n \times L \times (4d^2 + 2d^2) \times b, \tag{3}$$

$$\mathrm{MAV}_{\mathrm{moe}} = n \times L \times \left[K \times (4d_{\mathrm{e}}^2 + 2d_{\mathrm{e}}^2) + 4d_{\mathrm{r}}^2\right] \times b, \tag{4}$$

where b denotes the byte width of the storage data type. Applying Equations (3) and (4) to Qwen3-32B (dense, ∼32 GB/forward) and Qwen3-235B-A22B (MoE with eight activated experts, ∼51.8 GB/forward) shows that Qwen3-235B-A22B incurs approximately

61.9% higher MAV despite activating fewer parameters (22B vs. 32B), further shifting the bottleneck toward memory bandwidth.

Trend 2: Chain-of-thought reasoning paradigm.

Reasoning-oriented models [35,36] generate substantially longer output sequences, thereby increasing the time spent in the memory-bound decode phase. In traditional conversational workloads, the prompt is typically longer than the generated response, so prefill tends to dominate the overall inference time. By contrast, in reasoning workloads, the model may generate a large number of intermediate tokens before reaching the final answer, causing decode to become the dominant component of execution. Our workload analysis indicates that the prefill-to-decode time ratio shifts from approximately 3:1 on the ShareGPT [31] conversational dataset, which consists of multi-turn human-like dialogues, to 1:2 on the GSM8K [37] mathematical reasoning dataset, which contains grade school mathematical word problems. As a result, LLM inference becomes increasingly memory-intensive, since the decode phase is substantially more memory-bound than the prefill phase.

Trend 3: Growing hardware imbalance between compute and memory bandwidth.

GPU compute capability has grown far faster than memory bandwidth. As shown in Table 2, from NVIDIA V100 to H100, FP16 throughput increased by 8.6×, while HBM bandwidth grew by only 3.2×, reducing the bandwidth-to-FLOPS ratio from 64.3 to 24.0. This rising roofline ridge point means that an increasing fraction of LLM serving workloads falls into the memory-bound region.

Table 2. NVIDIA GPU architecture evolution: compute capability vs. memory bandwidth.

GPU	Year	TFLOPS (FP16)	HBM BW (GB/s)	BW/TFLOPS (FP16)
V100	2017	14	900	64.3
A100	2020	78	2039	26.1
H100	2022	120	2880	24.0

Implication:

The confluence of model sparsification, reasoning inference paradigm, and hardware trends is making LLM serving systematically more memory-bound—precisely the regime where GPU frequency reduction incurs minimal latency penalty yet yields proportional power savings. The energy-saving opportunity afforded by DVFS is therefore growing in both magnitude and prevalence.

3.4. Challenges

The above observations establish both the feasibility and the necessity of fine-grained, phase-aware DVFS for LLM serving. Motivated by these findings, our goal is to build an energy-efficient LLM serving framework that leverages fine-grained GPU DVFS to minimize inference energy consumption while strictly satisfying latency SLOs through iteration-level frequency optimization. Achieving this design goal raises three key technical challenges:

C1: Accurate iteration-level latency prediction under diverse workloads.

Avoiding the SLO violations exposed by Observation 2 requires predicting the latency impact of a candidate frequency f before it takes effect. The predictor must jointly account for inference phase, batch token count, KV cache size, and running request count yet remain lightweight enough for online use at iteration granularity—a regime in which

offline profiling cannot cover the full workload distribution and overly complex models themselves become a critical-path bottleneck.

C2: Dynamic frequency selection trade-off between SLO constraints and energy optimality.

The energy-optimal frequency f_{sweet} identified in Observation 1 and the SLO-feasible lower bound f_{slo} frequently conflict: as load rises or prefill work enters the batch, f_{sweet} falls below f_{slo} and the energy minimum becomes infeasible. Because both bounds shift continuously with request arrivals, batch composition, and phase transitions, no static or rule-based policy can stay near the joint optimum, calling for a controller that tracks these non-stationary boundaries online.

C3: Minimizing runtime overhead of energy-saving frameworks.

Iteration-level DVFS imposes two CPU-side costs on the per-iteration critical path: predictor inference, which evaluates every candidate frequency once per iteration, and `pyNVML` actuation, which adds 3–5 ms per invocation. Relative to a 20–100 ms decode iteration [38], these costs can erase most of the achievable energy savings unless the prediction and DVFS operations are overlapped with or amortized against the inference critical path.

4. System Design and Implementation

EcoInfer is designed to improve the energy efficiency of LLM serving through fine-grained GPU DVFS while preserving latency guarantees. Rather than replacing the underlying serving engine, EcoInfer is integrated into the inference engine as a lightweight control layer that operates in coordination with the scheduler and the GPU DVFS interface.

For ease of reference, Table 3 summarizes the main symbols used throughout this section.

Table 3. Notation used in the design of EcoInfer.

Symbol	Meaning
f	GPU core frequency
$f_{\min}, f_{\max}$	Minimum/maximum supported GPU core frequencies
f_{floor}	Sweet-spot-informed lower bound of the candidate frequency set
f_{slo}	Lowest frequency that still satisfies the iteration-level SLO budget
f_{sweet}	Energy-optimal frequency at the U-curve minimum
f^*	Target frequency selected by the controller for the current iteration
$\mathcal{F}$	Discretized candidate frequency set, $\{f_{\text{floor}}, f_{\text{floor}} + 100\,\text{MHz}, \ldots, f_{\max}\}$
x	Iteration feature vector (token counts and context lengths)
$\hat{T}(x, f)$	Predicted iteration latency under feature x and frequency f
$E(x, f)$	Iteration energy under feature x and frequency f
$P(f)$	GPU power as a function of frequency, $af^2 + bf + c$
B_{iter}	Iteration-level latency budget derived from the active SLO
N_p, N_d	Number of prefill/decode tokens in the current iteration
ℓ_i^p, ℓ_j^d	Per-request prefill/decode context lengths

Its design centers on a simple principle: frequency decisions should be made at the granularity of serving iterations, because both workload characteristics and latency sensitivity can vary substantially across iterations under continuous and mixed batching. To make such iteration-level control practical, EcoInfer combines three tightly coupled components: a latency predictor that estimates iteration latency under candidate frequencies, a controller that selects an SLO-compliant energy-efficient target frequency, and a low-overhead runtime path that minimizes the cost of online decision making and actuation. Figure 5 illustrates the overall architecture.

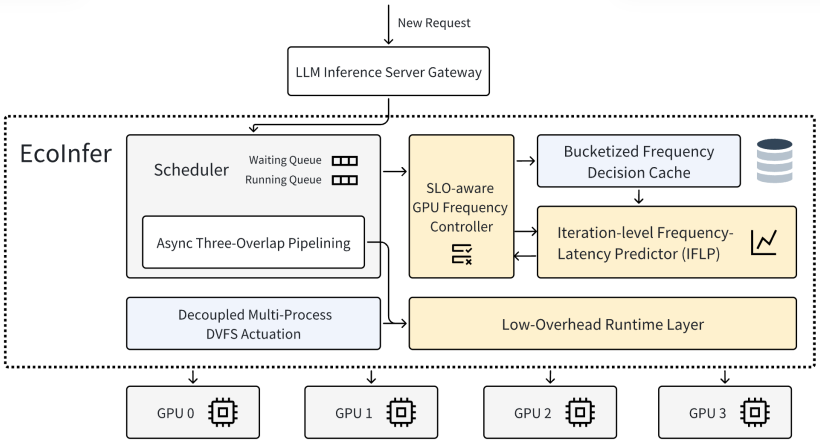


Figure 5. System overview of EcoInfer. EcoInfer is integrated into the LLM serving engine as a lightweight iteration-level DVFS control layer.

4.1. System Overview

EcoInfer sits between the LLM serving runtime and the GPU DVFS interface, augmenting the engine's continuous-batching workflow with iteration-level frequency control rather than altering its execution semantics. At the start of each iteration, the scheduler exposes the current batch composition, request and queue status, and iteration-level latency budget derived from the active SLO. Based on this state, EcoInfer selects a target GPU frequency and forwards the decision to the runtime layer for actuation through the DVFS interface.

We realize this scheduler-assisted, iteration-driven control loop with three cooperating components. The *Iteration-Level Frequency–Latency Predictor* (Section 4.2) maps the current batch state to a per-frequency latency estimate for every candidate in the frequency set. The *SLO-Aware GPU Frequency Controller* (Section 4.3) then uses these estimates together with the scheduler-provided iteration budget to select the minimum feasible frequency within a sweet-spot-guided candidate set. Finally, the *Low-Overhead Runtime Layer* (Section 4.4) keeps this loop affordable on the critical path through bucketized decision caching and asynchronous, decoupled DVFS actuation.

4.2. Iteration-Level Frequency–Latency Predictor

To enable fine-grained DVFS control, EcoInfer must estimate how much latency a serving iteration would incur under different GPU frequency settings before actually applying them. This capability is essential for the controller to evaluate candidate frequencies and select the lowest one that still satisfies the latency target. Direct online trial and error is impractical, since changing the frequency and measuring the resulting latency on the critical path would introduce prohibitive overhead. EcoInfer therefore incorporates an *Iteration-Level Frequency–Latency Predictor (IFLP)*, a lightweight machine-learning-driven runtime module that predicts the execution latency of the current iteration under a specified GPU frequency.

Designing such a predictor is non-trivial because modern LLM serving engines no longer execute iterations in a strictly phase-separated manner. With continuous batching and mixed batching, prefill requests and decode requests can be packed into the same iteration and jointly executed by the attention and other forward operators. As a result, an iteration cannot be accurately characterized by a simple prefill/decode label. Its latency depends instead on the fine-grained composition of the batched workload, including how many prefill and decode tokens are processed together and how much context each part carries. This property makes coarse phase-based prediction insufficient for runtime DVFS decisions.

EcoInfer addresses this challenge by modeling latency at the iteration granularity. The predictor takes as input the current iteration features together with a target GPU frequency and outputs the predicted execution latency for that frequency. Our feature design follows a systems-oriented principle: the predictor should capture the minimal workload description that determines operator tensor shapes, since these shapes directly govern the amount of computation and memory traffic in the forward pass and thus determine the iteration's sensitivity to GPU frequency scaling. Based on this principle, we use four iteration-level features—the number of prefill tokens, the number of decode tokens, the prefill context length, and the decode context length—together with the candidate GPU frequency f. These features jointly describe the token composition and context state of a mixed batch. In particular, the numbers of prefill and decode tokens determine the amount of work contributed by each request type in the current iteration, while the corresponding context lengths determine the effective sequence dimensions of attention and related operators. Together, they characterize the iteration's computational intensity, memory access behavior, and frequency sensitivity without requiring an explicit binary phase classification. This choice is also consistent with recent LLM inference simulators such as Vidur [39], which estimate iteration runtime from analogous token-count and per-request context-length features.

The predictor must satisfy two competing requirements. On the one hand, prediction error must be low, because inaccurate latency estimates may cause the controller to select an overly aggressive frequency and violate the SLO or choose an unnecessarily high frequency and miss energy-saving opportunities. On the other hand, prediction overhead must be minimal, because the predictor is invoked online in the serving path and its own cost directly offsets the benefit of fine-grained DVFS control. For this reason, EcoInfer adopts a lightweight machine learning model rather than either a handcrafted analytical model or a heavyweight neural predictor. Analytical models are difficult to make accurate under mixed-batch execution, where kernel behavior depends on the interaction between token composition, context length, and device frequency. Conversely, heavier learned models can improve expressiveness but incur excessive runtime overhead for online use.

We explored several candidate models, including Ridge [40], Random Forest [41] and MLP [42], and ultimately chose to train IFLP using XGBoost [43]. This XGBoost-based IFLP provides the best balance between prediction accuracy and inference cost in our setting. To train IFLP, we first collect offline profiling data by executing representative iterations under different GPU frequencies and recording their observed latencies. We then use these profiled samples to pretrain the model before deployment. At runtime, EcoInfer only performs lightweight inference: given the current iteration features and candidate frequencies, IFLP returns the estimated latency of that iteration under the specified frequency. In our experiments, this design achieves a mean absolute percentage error (MAPE) of only 1.13% while keeping predictor inference latency below 4 ms. Section 5 further compares IFLP with alternative models and evaluates the trade-off between prediction accuracy and runtime overhead.

Overall, IFLP serves as the foundation of EcoInfer's control loop. By transforming the mixed-batch runtime state into accurate and low-overhead latency estimates, IFLP allows the controller to reason about the feasibility of different frequency choices without expensive online probing, thereby enabling practical SLO-aware DVFS at iteration granularity.

4.3. SLO-Aware GPU Frequency Controller

Given the latency predictor in Section 4.2, the next question is how to translate predicted latencies into a concrete GPU frequency decision for each serving iteration. EcoInfer addresses this problem with an *SLO-Aware GPU Frequency Controller*, whose goal is to

minimize iteration energy while preserving latency guarantees. At a high level, the controller receives the current iteration features from the scheduler, queries the predictor over a discrete set of candidate frequencies, and selects the lowest frequency that is still predicted to satisfy the iteration-level SLO budget. Figure 6 illustrates this decision procedure. This design directly operationalizes the observations in Sections 3.1 and 3.2 that DVFS for LLM serving is beneficial only when frequency selection is both workload-aware and latency-constrained.

Formally, let x denote the feature vector of the current iteration, let $\hat{T}(x, f)$ denote the latency predicted by IFLP under GPU frequency f, and let B_{iter} denote the latency budget assigned to this iteration by the scheduler. The controller solves the following constrained optimization problem:

$$f^* = \arg\min_{f \in \mathcal{F}} E(x, f) \quad \text{s.t.} \quad \hat{T}(x, f) \leq B_{\text{iter}}, \tag{5}$$

where $\mathcal{F}$ is the candidate frequency set and $E(x, f)$ is the iteration energy under workload state x and frequency f. Rather than explicitly estimating $E(x, f)$ online, EcoInfer leverages the U-shaped energy–frequency relationship established in Observation 1. Specifically, offline profiling reveals a sweet-spot-informed lower bound f_{floor}, below which further frequency reduction tends to incur disproportionate latency growth and may even increase total energy. We therefore restrict the search space to the interval $[f_{\text{floor}}, f_{\max}]$ and discretize it using 100 MHz steps:

$$\mathcal{F} = \{f_{\text{floor}}, f_{\text{floor}} + 100\,\text{MHz}, \ldots, f_{\max}\}. \tag{6}$$

Within this range, lower frequency generally implies lower power without entering the clearly energy-inefficient regime on the left side of the U-curve. As a result, the controller can simplify Equation (5) into the following selection rule:

$$f^* = \min\{f \in \mathcal{F} \mid \hat{T}(x, f) \leq B_{\text{iter}}\}. \tag{7}$$

That is, EcoInfer selects the minimum feasible frequency among all candidates whose predicted latency satisfies the current iteration budget. Because the candidate set is already pruned to the sweet-spot-informed region, this minimum feasible frequency also serves as the energy-optimal choice under the SLO constraint.

A key challenge is that the latency constraint is not fixed across all iterations. Instead, the scheduler derives and passes an iteration-level budget B_{iter} according to the composition of the current batch. If the batch contains only decode requests, the controller uses the token-by-token latency budget, i.e., the TBT SLO, as B_{iter}. If the batch contains only prefill requests, B_{iter} is set to the TTFT SLO. For mixed batches containing both prefill and decode requests, EcoInfer conservatively sets

$$B_{\text{iter}} = \min(\text{TBT SLO}, \text{TTFT SLO}), \tag{8}$$

thereby ensuring that the selected frequency remains compatible with both latency requirements. Whenever TTFT and TBT disagree on which is more constraining, the controller deliberately honors the tighter of the two as B_{iter}. This design allows the controller to operate on a unified abstraction—an iteration-level latency budget—while still respecting the heterogeneous QoS semantics of different request types.

The controller also incorporates a simple but important load-aware safeguard. When the scheduler indicates that the waiting queue is non-empty, the system is already under high load and should prioritize throughput and latency headroom over energy reduction. In

this case, EcoInfer bypasses the prediction-and-selection procedure entirely and directly sets the target frequency to $f_{\max}$. This policy prevents the controller from spending optimization effort in clearly overloaded regimes where aggressive energy saving is unlikely to be beneficial and may instead exacerbate queueing delays.

Under normal load, the controller evaluates the candidate set in a predictor-guided manner. For the current iteration feature vector x, EcoInfer forms a batch of predictor inputs by pairing x with all candidate frequencies in $\mathcal{F}$. The predictor then performs batched inference and returns a latency estimate for each candidate frequency. Compared with evaluating candidates one by one, batched prediction reduces decision overhead and fits naturally with the inference pipeline of IFLP. Once all predicted latencies are obtained, the controller applies Equation (7) to select the target frequency. If no candidate frequency satisfies the latency budget, EcoInfer again falls back to $f_{\max}$ to preserve service quality.

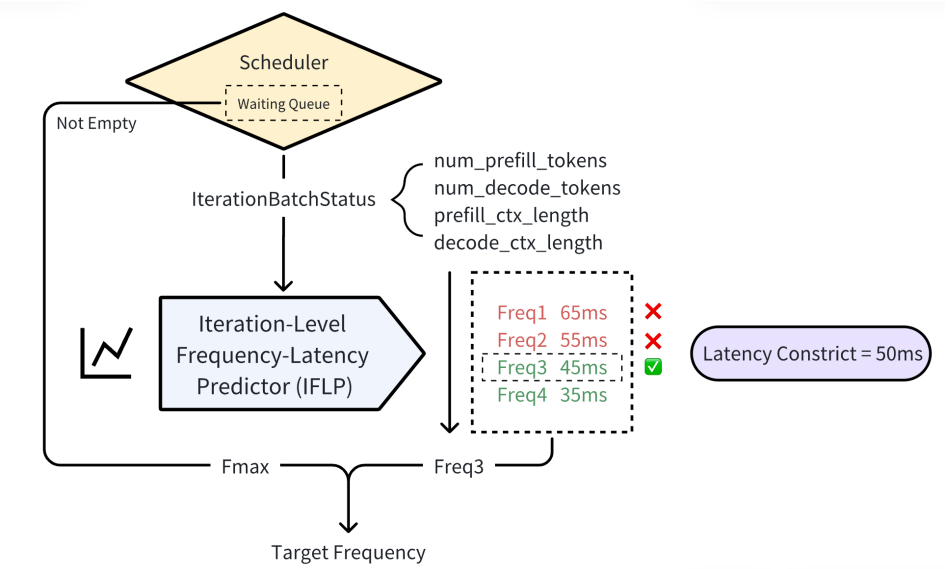


Figure 6. Frequency selection logic of the SLO-Aware GPU Frequency Controller.

Finally, after the target frequency is determined, the controller passes it to the runtime layer for enforcement through the GPU DVFS interface. In our implementation, frequency actuation is performed via `pyNVML`. However, directly invoking frequency changes in the critical path can introduce non-negligible CPU-side overhead. We therefore separate the decision logic in this subsection from the execution optimization in Section 4.4, where we describe how EcoInfer reduces and overlaps the overhead of frequency prediction and actuation.

Overall, the SLO-Aware GPU Frequency Controller serves as the decision-making core of EcoInfer. It bridges the predictor's latency estimates and the runtime DVFS mechanism through a simple yet effective policy: aggressively preserve performance under high load, and otherwise select the minimum feasible frequency within a sweet-spot-guided candidate range. This policy enables EcoInfer to convert iteration-level workload information into energy-efficient, SLO-compliant frequency decisions in real time.

4.4. Low-Overhead Runtime Optimizations for Iteration-Level Control

While the predictor and controller in Sections 4.2 and 4.3 enable accurate and SLO-aware frequency selection, they do not by themselves make iteration-level DVFS practical. A straightforward synchronous implementation would still place non-trivial CPU-side control overhead on the critical path of every serving iteration. To address this challenge, EcoInfer introduces two complementary runtime optimizations: a Bucketized Frequency Decision Cache that reduces how often frequency prediction must be invoked, and an

Asynchronous Pipelining and Decoupled Actuation mechanism that hides the residual miss-path overhead behind GPU execution.

4.4.1. Critical-Path Overhead in Naive Synchronous Control

In a naive synchronous design, each iteration executes the following stages in sequence:

$$T_{\text{serial}} = T_{\text{sched_cpu}} + T_{\text{freq_decision_cpu}} + T_{\text{DVFS_cpu}} + T_{\text{forward_gpu}}, \tag{9}$$

where $T_{\text{sched_cpu}}$ denotes batch scheduling overhead on the CPU, $T_{\text{freq_decision_cpu}}$ denotes the time for predictor-guided frequency selection, $T_{\text{DVFS_cpu}}$ denotes the CPU-side cost of invoking the DVFS interface, and $T_{\text{forward_gpu}}$ denotes the GPU forward execution time of the current iteration. In our implementation, these components are approximately 1 ms, 3 ms, 5 ms, and 40–500 ms, respectively. As a result, frequency decision and DVFS actuation alone account for roughly 15% of the iteration latency. More importantly, during this CPU-side control interval, the system cannot launch the next forward pass on the GPU, which delays forward progress and significantly reduces the benefit of fine-grained DVFS. This observation implies that accurate prediction and sound control logic are not sufficient on their own. Iteration-level frequency control must also be implemented with low runtime overhead.

4.4.2. Bucketized Frequency Decision Cache

The first source of overhead comes from repeated predictor invocation during frequency selection. EcoInfer reduces this cost by exploiting a key property of continuous batching: iteration-level temporal locality. In steady-state decoding, adjacent iterations often have highly similar workload characteristics. For example, when the set of running requests remains unchanged, decode requests typically differ only by a small increase in context length between successive iterations. Substantial workload shifts mainly occur when a new request enters prefill or an existing request finishes generation. Therefore, many consecutive iterations exhibit similar frequency requirements, making direct reuse of recent decisions feasible.

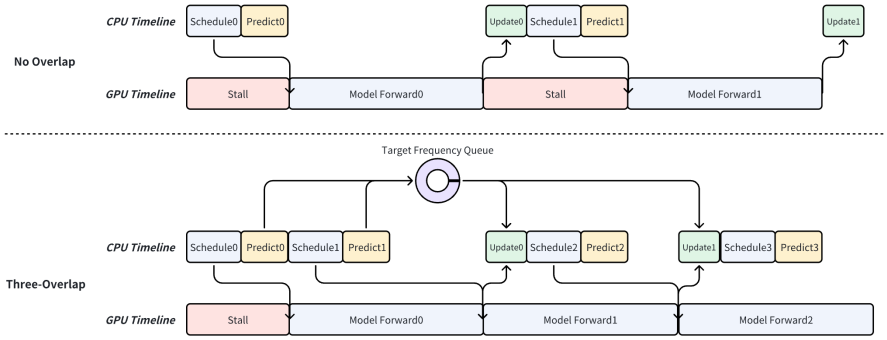
Based on this observation, EcoInfer introduces a bucketized frequency decision cache. The cache key is constructed by bucketizing the four iteration features, the number of prefill tokens, the number of decode tokens, the prefill context length, and the decode context length, using a fixed bucket size. The cached value is the target frequency selected by the controller for the corresponding bucketized workload state. We manage the cache using an LRU policy so that recently observed iteration patterns remain available for reuse. On a cache hit, EcoInfer directly reuses the cached target frequency and skips the predictor-guided selection procedure, thereby eliminating most of the decision overhead on the hit path.

A key design trade-off lies in the choice of bucket granularity. If the bucket size is too large, iterations with noticeably different workload characteristics may map to the same cache entry, which can reduce decision fidelity and potentially hurt either SLO compliance or energy efficiency. If the bucket size is too small, cache reuse opportunities diminish and the hit rate becomes too low to provide meaningful acceleration. In our deployment, we set the bucket size to 128 and keep the most recent 100 cache entries. This configuration provides a good balance between reuse and fidelity, achieving an average cache hit rate of approximately 73% in practice. We evaluate the impact of cache configuration and hit behavior in Section 5.

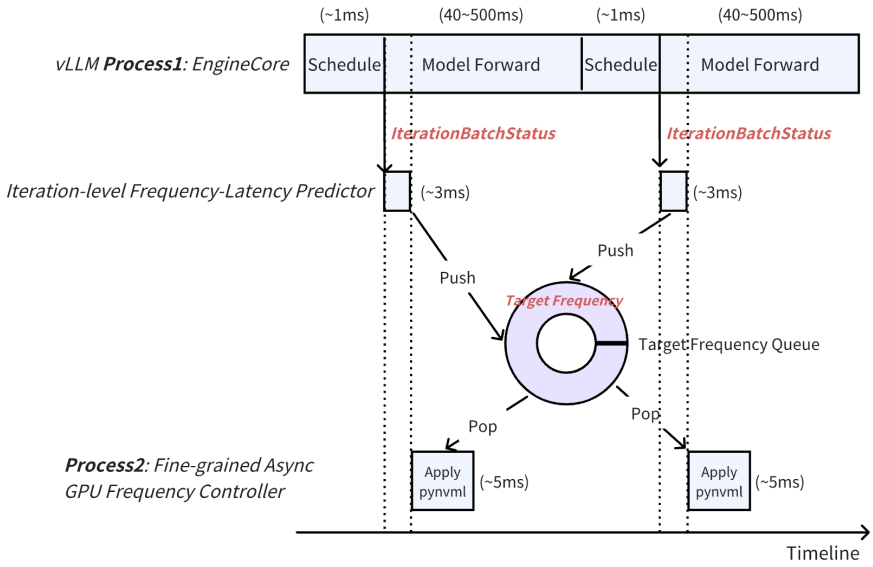
4.4.3. Asynchronous Pipelining and Decoupled DVFS Actuation

Although the decision cache eliminates most predictor invocations, cache misses still incur non-negligible CPU-side overhead. Moreover, even after frequency selection is completed, directly applying the target frequency through `pyNVML` can block the inference process and further extend the critical path. To address these remaining costs, EcoInfer employs asynchronous pipelining and decoupled DVFS actuation.

The core idea is to overlap next-iteration control preparation with current-iteration GPU execution. While the GPU is executing the forward pass of the current iteration, the CPU simultaneously performs scheduling and, if needed, predictor-guided frequency selection for the next iteration. The selected target frequency is then pushed into a target-frequency queue. Once the current GPU forward pass completes, the runtime can immediately consume the prepared decision from the target-frequency queue instead of waiting for scheduling and control logic to execute serially. This pipelined design removes most of the scheduling and decision latency from the GPU critical path and thereby minimizes GPU idle time and improves GPU utilization, as illustrated in Figure 7a.



(**a**) Asynchronous three-overlap pipelining.



(**b**) Decoupled two-process DVFS actuation.

Figure 7. Low-overhead runtime design of EcoInfer. (**a**) Three-overlap pipelining hides CPU-side scheduling and frequency prediction behind GPU execution, eliminating the GPU stalls present in naive control. (**b**) A dedicated frequency-controller process applies DVFS decisions asynchronously via `pyNVML`, decoupling the blocking actuation call from the inference critical path.

EcoInfer further decouples DVFS actuation from the inference process by placing frequency enforcement in a separate CPU process. Specifically, as shown in Figure 7b, the inference runtime and the frequency controller run independently, and the DVFS process retrieves target frequencies from the queue and invokes `pyNVML` asynchronously. This separation prevents the blocking behavior of the frequency-setting API from interfering with the main inference process. As a result, scheduling, frequency decision, and DVFS actuation no longer serialize with GPU computation in the common case.

Together, these two optimizations make iteration-level control practical. The decision cache reduces how often predictor-guided search is required, while asynchronous pipelining and decoupled actuation hide the residual miss-path overhead behind ongoing GPU execution. In combination, they transform iteration-level DVFS from a theoretically effective but operationally expensive mechanism into a low-overhead runtime control loop that can be deployed in latency-sensitive LLM serving systems.

5. Evaluation

We evaluate EcoInfer on a single-node LLM serving testbed to answer the following questions: (1) Can EcoInfer improve energy efficiency while preserving latency guarantees under realistic online workloads? (2) How do the benefits vary across dense and MoE models and across different request rates? (3) Does EcoInfer adapt GPU frequency in a manner consistent with runtime workload dynamics? (4) How much do the predictor design and the low-overhead runtime optimizations contribute to the final system efficiency?

5.1. Experimental Setup

5.1.1. Hardware and Software

All experiments are conducted on a high-performance inference server equipped with two Intel Xeon Platinum 8369B CPUs and eight NVIDIA A100 GPUs with 80 GB HBM per GPU. The GPUs are interconnected through NVLink, and the system provides 1 TB DRAM and 3.5 TB NVMe SSD storage. The software stack runs on Ubuntu 22.04 LTS, with Python v3.12, PyTorch v2.9.1, NVIDIA driver 535.129.03, and CUDA v12.2. EcoInfer is implemented on top of vLLM [15], one of the most widely used open-source LLM serving engines.

5.1.2. Models and Workload

We evaluate three representative models: Qwen3-8B and Qwen3-32B as dense models and Qwen3-30B-A3B as an MoE model [30]. For model deployment, Qwen3-32B is served with tensor parallelism [44] across two GPUs (TP = 2), while Qwen3-8B and Qwen3-30B-A3B are each served on a single GPU with TP = 1. Unless otherwise stated, we use the ShareGPT [31] trace to emulate realistic online serving traffic. We sweep the request arrival rate (RPS) to study both light-load and high-load operating regimes.

5.1.3. Baselines and Metrics

For end-to-end evaluation, our primary baseline is vLLM, which runs vLLM under the default NVIDIA GPU DVFS policy. This policy generally favors higher GPU frequencies when requests are active but does not incorporate any explicit energy-aware DVFS optimization. We do not include direct comparisons against DynamoLLM [11] or ThrottLL'eM [12], which are discussed in Section 2, because their implementations are not publicly available, preventing faithful reproduction and fair evaluation on our testbed.

For module-level ablations, we use targeted comparisons rather than a single unified baseline. In Section 5.5, we compare multiple candidate prediction models, including Ridge [40], Random Forest [41], MLP [42], HistGBDT [45], and our IFLP, which is trained us-

ing an XGBoost-based model. In Section 5.6, we evaluate the runtime optimizations in two steps: we first compare the EcoInfer system with its bucketized frequency-decision cache enabled versus the system with its cache disabled to isolate cache effectiveness, and then we compare No Overlap and Overlap in order to isolate the benefit of asynchronous pipelining and decoupled DVFS actuation.

Depending on the experiment, we report the average per-request GPU energy consumption (J), energy efficiency in TPJ, SLO attainment rate, throughput, predictor accuracy metrics (MAE, P90/P99 MAE, RMSE, R^2, and MAPE), predictor inference latency, and execution-time breakdown. Unless otherwise stated, all subsequently reported energy numbers refer to this per-request average, computed as the total GPU energy consumed over the entire trace divided by the total number of completed requests; this normalization avoids confounding the metric with trace duration or request volume across different RPS settings and SLO configurations. Following the system design in Section 4, we use TTFT and TBT as the two latency SLO dimensions. Because latency characteristics vary across model sizes and architectures, we do not impose a single unified TTFT/TBT target on all settings. Instead, we use model-specific TTFT and TBT thresholds for each model configuration and measure SLO attainment against the corresponding targets. Specifically, the SLO targets are set as follows: for Qwen3-8B, TTFT = 150 ms and TBT = 50 ms; for Qwen3-32B, TTFT = 250 ms and TBT = 100 ms; and for Qwen3-30B-A3B, TTFT = 200 ms and TBT = 80 ms.

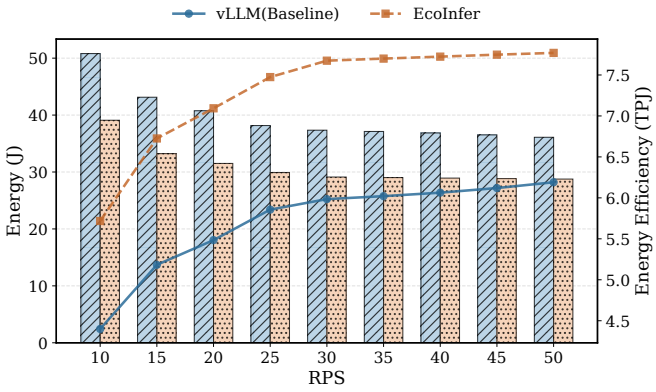
5.2. Overall Energy Efficiency

We first evaluate whether EcoInfer delivers end-to-end energy savings under realistic online serving workloads. Figure 8 reports the total energy consumption and TPJ of EcoInfer and baseline vLLM across different request rates for the two dense models and the MoE model. Across all three models, EcoInfer consistently reduces energy consumption while improving TPJ efficiency. Overall, EcoInfer achieves up to 25.4% energy savings and 21.5% average energy savings, corresponding to a 1.28× average improvement in TPJ. At the model level, EcoInfer reduces energy by up to 23.1%, 20.8%, and 25.4% on Qwen3-8B, Qwen3-32B, and Qwen3-30B-A3B, respectively, while the corresponding peak TPJ improvements reach 30%, 26.26%, and 34%.

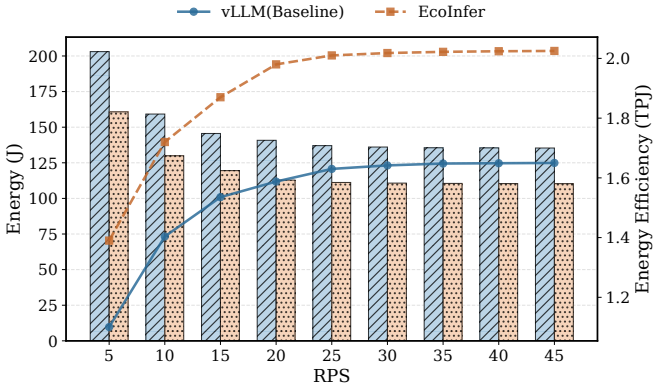
Two observations are worth highlighting. First, the improvement persists across a broad range of RPS values rather than appearing only in a narrow operating regime, indicating that EcoInfer is robust to varying online load. Moreover, the gain is more pronounced under light load, where the system has more performance slack under the same SLO target and therefore more power redundancy that can be converted into energy savings. For example, on Qwen3-8B, EcoInfer improves energy efficiency by 30% at low load (RPS = 10) compared with 25.5% at high load (RPS = 50).

Second, the MoE model shows larger energy-efficiency gains than the dense models. Across all evaluated RPS values, the Qwen3-8B dense model improves energy efficiency by 25.5–30%, whereas Qwen3-30B-A3B improves energy efficiency by 30–34%. This result is consistent with Observation 3 in Section 3: as inference becomes more memory-bound, the latency penalty of reducing GPU frequency becomes smaller, leaving more room for energy savings.

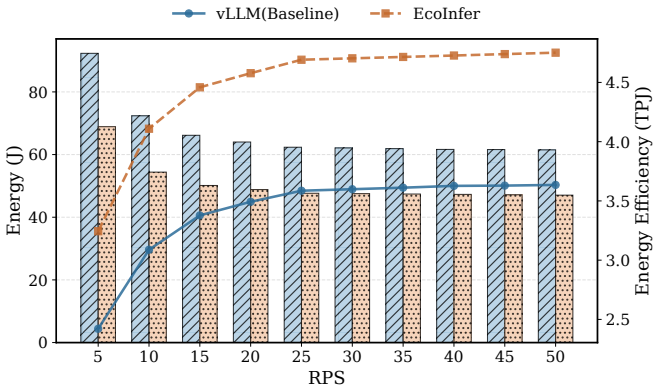
Energy savings would be of limited practical value if they came at the cost of degraded service quality. We therefore compare the SLO attainment rate of EcoInfer and vLLM in Figure 9. Across all models and request rates, EcoInfer maintains an SLO attainment rate that is nearly unchanged relative to the default vLLM baseline, with the absolute gap never exceeding 5%. This result shows that EcoInfer successfully converts workload-dependent slack into energy savings without materially compromising latency guarantees.



(**a**) Qwen3-8B.



(**b**) Qwen3-32B.



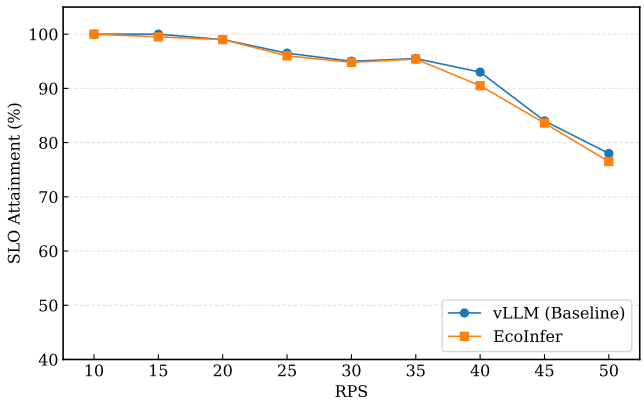
(**c**) Qwen3-30B-A3B.

Figure 8. Energy consumption and energy efficiency across request rates on the ShareGPT trace. (**a**) Qwen3-8B, (**b**) Qwen3-32B, and (**c**) Qwen3-30B-A3B. Blue/orange bars denote the average per-request GPU energy consumption of vLLM/EcoInfer; lines denote energy efficiency in TPJ. EcoInfer is compared with the default-DVFS vLLM baseline, with the largest gains observed under light load and on the MoE model.

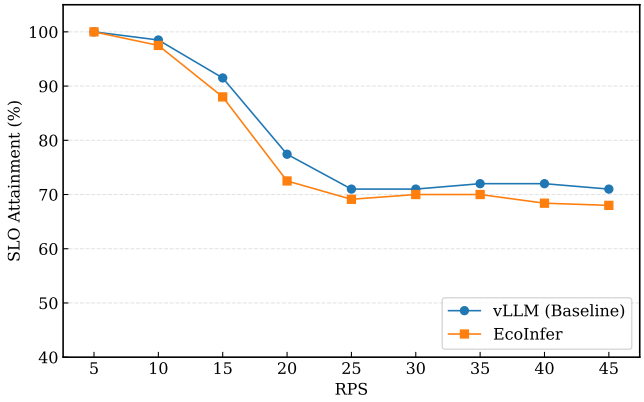
5.3. Runtime Behavior Analysis

To better understand how EcoInfer behaves online, we examine the time-varying GPU frequency trace during serving. Figure 10 shows that EcoInfer does not keep the GPU at a fixed operating point. Instead, it continuously adapts the frequency to the runtime workload state. When the batch is decode-heavy and latency slack is available, EcoInfer lowers the GPU frequency to reduce power consumption. When the workload becomes

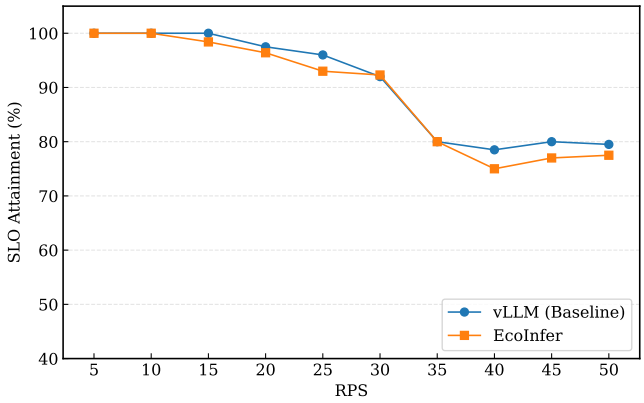
more latency-sensitive—for example, due to prefill-heavy iterations or increased system pressure—EcoInfer raises the frequency accordingly.



(**a**) Qwen3-8B.



(**b**) Qwen3-32B.



(**c**) Qwen3-30B-A3B.

Figure 9. SLO attainment rate across request rates on the ShareGPT trace. (**a**) Qwen3-8B, (**b**) Qwen3-32B, and (**c**) Qwen3-30B-A3B. The metric reports the percentage of requests satisfying both TTFT and TBT constraints. EcoInfer stays within a few percentage points of the default-DVFS vLLM baseline, indicating that its energy savings do not materially erode latency guarantees.

This behavior is important for two reasons. First, it validates that EcoInfer is indeed performing fine-grained runtime adaptation rather than converging to a nearly static frequency. Second, it is qualitatively consistent with the design rationale in Section 4: the controller uses a scheduler-visible state and predictor-guided selection to exploit energy-saving opportunities during decode-dominated periods while preserving latency headroom

for more demanding iterations. In other words, the frequency trace provides an operational view of how the predictor and controller translate workload dynamics into online DVFS decisions.

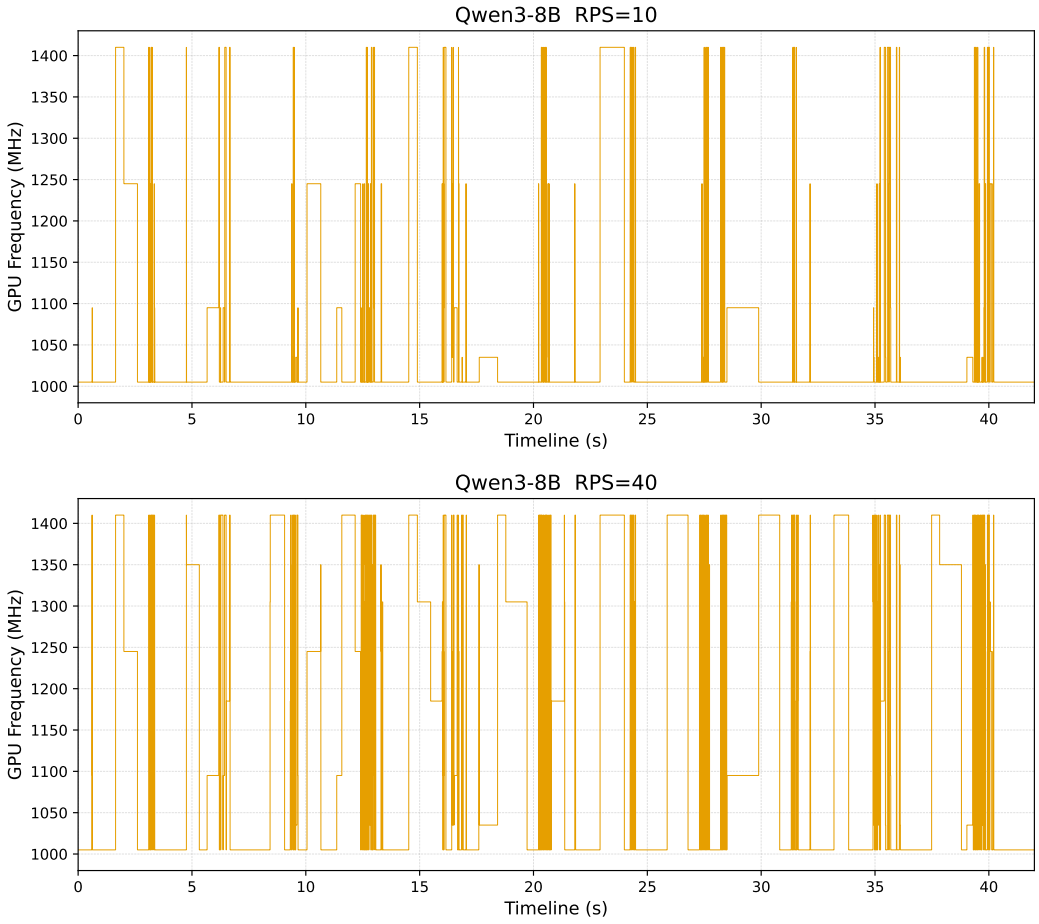


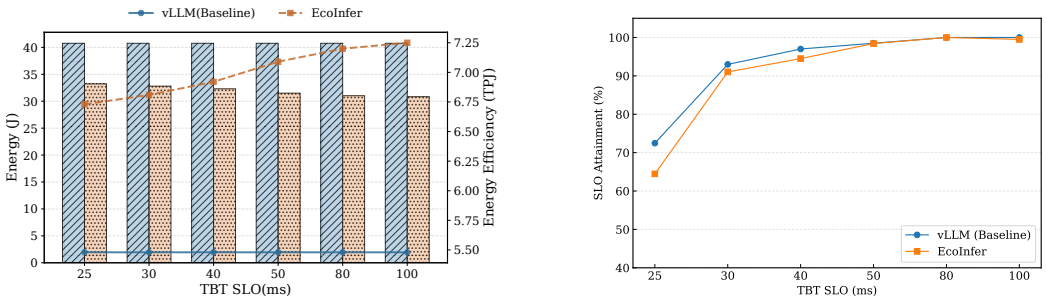
Figure 10. GPU core frequency trace over wall-clock time under EcoInfer for Qwen3-8B on ShareGPT, shown for two arrival rates. Each point reports the frequency selected by the SLO-aware controller for that iteration. The trace illustrates that EcoInfer continuously adapts the GPU frequency to the current serving state—lowering it during decode-heavy or low-pressure intervals and raising it for prefill-heavy or high-pressure iterations—rather than converging to a single static operating point. The lower-RPS run spends substantially more time at reduced frequencies, which directly explains the larger energy savings observed under light load.

Moreover, as shown in Figure 10, the effect of such adaptive control becomes more pronounced under lighter request arrival rates. For Qwen3-8B, when the workload RPS is 10, EcoInfer spends substantially more time operating at relatively low frequency levels than it does under RPS = 40, thereby achieving greater energy savings. This observation further demonstrates that EcoInfer can dynamically infer the current workload pressure and adjust its operating point accordingly rather than relying on a fixed policy across different serving intensities.

5.4. Sensitivity to SLO Configuration

EcoInfer's frequency-selection rule (Equation (7)) depends on the iteration-level latency budget B_{iter} derived from the active SLO. To probe this dependence, we fix the model to Qwen3-8B and the arrival rate to RPS $=$ 20 on the ShareGPT trace, keep the TTFT SLO at its default value, and sweep the TBT SLO across $\{25, 30, 40, 50, 80, 100\}$ ms. Figure 11 reports the resulting energy/TPJ and SLO attainment behavior.

Two trends emerge. First, energy savings grow monotonically with looser TBT SLOs. As the TBT SLO loosens from 25 ms to 100 ms, EcoInfer's energy drops from approximately 33.21 J to approximately 30.84 J, while the vLLM baseline stays flat near 40.76 J (it always runs at $f_{\max}$), so energy savings relative to vLLM grow from approximately 18.5% to 24.3% and TPJ improves accordingly. This is exactly what Equation (7) predicts: a larger B_{iter} admits a lower minimum feasible frequency, so the controller drives more iterations toward f_{floor}. Second, SLO attainment tracks the vLLM baseline closely across the practical SLO range. For any TBT $\geq$ 30 ms, the absolute attainment gap between EcoInfer and vLLM is at most 3%. Only at the extremely tight TBT $=$ 25 ms does the gap widen (vLLM $\approx$ 72.5% vs. EcoInfer $\approx$ 64.5%). However, at this point the SLO is intrinsically infeasible even at $f_{\max}$, so this regime lies outside the practical operating envelope. The load-aware fallback to $f_{\max}$ described in Section 4.3 ensures that worst-case attainment remains close to the vLLM baseline rather than degrading further.



(**a**) Energy and TPJ vs. TBT SLO. (**b**) SLO attainment vs. TBT SLO.

Figure 11. Sensitivity to TBT SLO on Qwen3-8B at RPS $=$ 20 (ShareGPT). (**a**) Blue/orange bars denote the average per-request GPU energy consumption of vLLM/EcoInfer; lines denote energy efficiency in TPJ. (**b**) SLO attainment (%, higher is better).

5.5. Ablation: Predictor Design

We next evaluate the design choice of IFLP. The key question is whether EcoInfer can achieve sufficiently high prediction accuracy without introducing excessive online overhead. To ensure that this evaluation is representative of practical deployment, we train and test all predictor models using samples collected from online serving runs driven by the ShareGPT workload, with an 80/20 split for training and testing.

Figure 12 and Table 4 compare candidate models in terms of mean absolute error (MAE), RMSE, R^2, and MAPE. The results show that IFLP provides the best overall accuracy–efficiency trade-off among the evaluated predictors. Compared with Random Forest, which achieves the lowest MAE, IFLP incurs only a 0.05 ms higher MAE while avoiding the substantial online overhead of the heavier model. This gap in deployment cost is important because Random Forest exhibits an average inference latency above 20 ms, making it difficult to use in a practical online serving loop. By contrast, IFLP reduces inference latency by 88% relative to Random Forest, with mean and P90 inference latencies both below 6 ms. In addition, the P99 prediction error of IFLP remains well controlled below 3 ms, which is critical because underestimating latency at the tail can directly translate into overly aggressive frequency choices and potential SLO violations.

Figure 13 further isolates model inference latency. At the other end of the design space, lightweight predictors such as Ridge and MLP-Light incur low inference latency but deliver substantially worse prediction accuracy. Their average MAE all exceeds 1.5 ms, and such large prediction errors can materially distort the frequency decision and thus reduce SLO attainment in practice.

Table 4. Prediction accuracy comparison of models on the test set.

Model	RMSE ↓	R^2 ↑	MAPE (%) ↓
Ridge	2.407	0.955	7.147
MLP-Light	2.331	0.956	7.041
HistGBDT	2.525	0.954	1.698
IFLP (ours)	**1.551**	**0.961**	**1.131**
Random Forest	1.427	0.962	0.815

Note: ↓ indicates lower is better, and ↑ indicates higher is better. Underline denotes the best result, and bold denotes the second best.

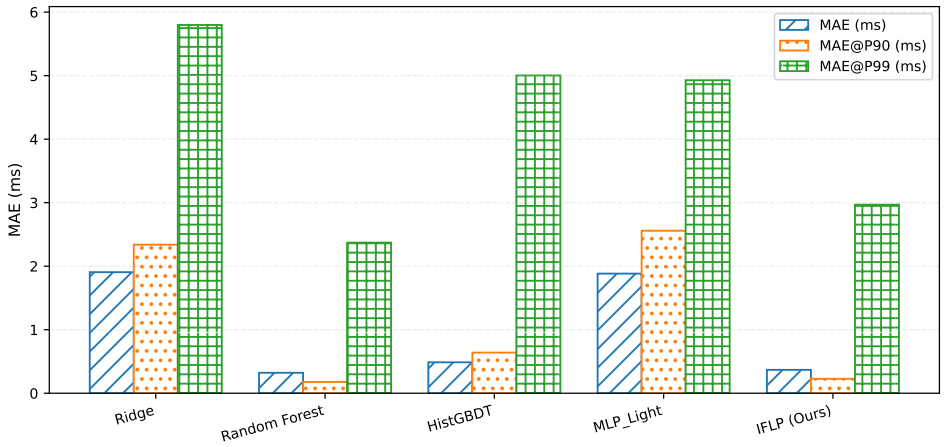


Figure 12. Prediction accuracy of candidate latency predictors. We compare MAE, P90 MAE, and P99 MAE across alternative models.

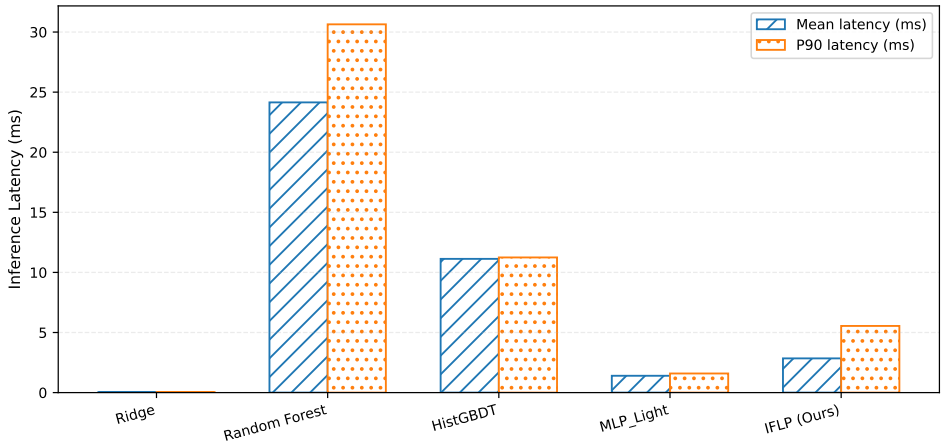


Figure 13. Inference latency of candidate prediction models. We report the mean and P90 inference latencies. Low online predictor cost is essential for practical iteration-level DVFS.

Taken together, these results show that IFLP achieves the most favorable balance between prediction accuracy and online overhead, which is precisely the design objective of Section 4.2.

5.6. Ablation: Cache and Asynchronous Overlap

We finally evaluate the two runtime optimizations introduced in Section 4.4: the bucketized frequency-decision cache and the asynchronous pipeline runtime design. Rather than comparing all runtime variants against a single baseline, we isolate the contribution of each mechanism separately. We first measure the benefit of the bucketized frequency-decision cache alone, and then we disable the cache to isolate how asynchronous pipelining and decoupled DVFS actuation reduce the remaining control-path overhead.

5.6.1. Cache Effectiveness

We first isolate the effect of the bucketized frequency-decision cache under synchronous control. Unless otherwise stated, this experiment uses Qwen3-8B on the ShareGPT trace at a fixed arrival rate of 40 RPS, and compares EcoInfer with the bucketized frequency-decision cache enabled versus disabled while keeping all other components unchanged.

Figure 14 compares the runtime behavior of EcoInfer with and without the decision cache. The cache achieves a 73% hit rate over time, confirming that adjacent iterations indeed exhibit strong temporal locality under continuous batching. More importantly, this temporal locality translates into throughput gains because, on a cache hit, EcoInfer can directly reuse the previously selected frequency and skip the predictor-guided frequency search. Under synchronous control, predictor invocation lies on the iteration critical path. Reducing its frequency therefore shortens the average CPU-side control latency and allows the next GPU forward pass to start earlier. As these savings accumulate over many steady-state iterations, the system achieves a higher effective iteration processing rate and thus a 5.15% end-to-end throughput improvement.

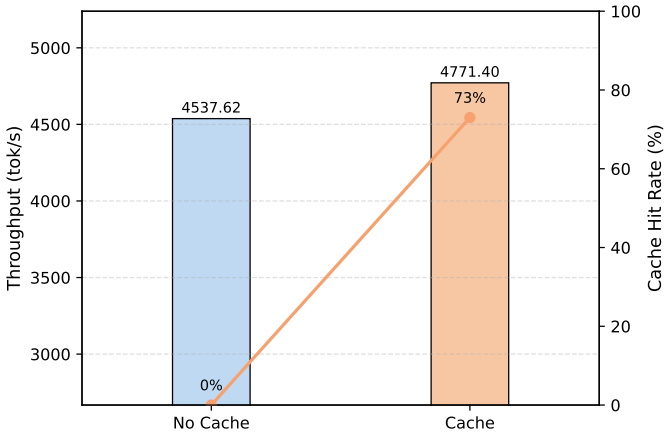


Figure 14. Effectiveness of the bucketized frequency-decision cache. Bars denote throughput, and the solid line denotes cache hit rate. The cache improves throughput while achieving a high decision-reuse rate over time.

At the same time, the figure also shows that the cache does not eliminate control overhead entirely. Approximately 27% of iterations still fall onto the miss path and require full predictor-guided selection. Moreover, GPU frequency actuation itself still incurs a non-negligible CPU-side cost through `pyNVML` regardless of whether the frequency decision is obtained from the cache or from online prediction. These two residual overheads—miss-path decision latency and per-iteration DVFS actuation cost—motivate the asynchronous runtime design evaluated next.

5.6.2. Asynchronous Overlap Effectiveness

We next isolate the benefit of the asynchronous runtime design by disabling the frequency-decision cache and comparing a purely synchronous control flow (No Overlap) against the full asynchronous runtime with pipelining and decoupled DVFS actuation (Overlap). Unless otherwise stated, this experiment uses Qwen3-8B on the ShareGPT trace at a fixed arrival rate of 40 RPS. By removing the cache, every iteration must execute the full control path, making the impact of asynchronous overlap directly visible.

Figure 15 reports the execution time breakdown for the two configurations. In the No Overlap design, the scheduler, the IFLP predictor, and the `pyNVML` actuation step are all exposed on the critical path before GPU execution can proceed. As a result, the GPU-visible stall reaches 6.83 s, accounting for 27.7% of the total 24.62 s execution time. Within this

exposed overhead, the scheduler contributes 1.30 s (5.3%), IFLP prediction contributes 1.47 s (6.0%), and the application of the DVFS change through `pyNVML` adds another 3.57 s (14.5%), showing that CPU-side control can significantly delay GPU forward progress.

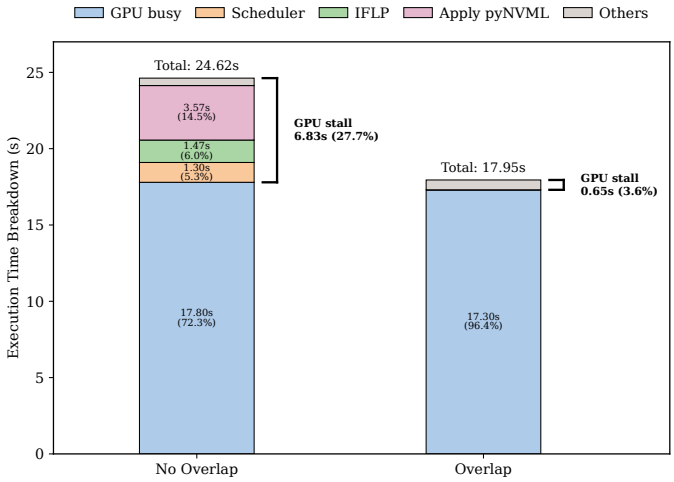


Figure 15. End-to-end execution-time breakdown of EcoInfer with the frequency-decision cache disabled, showing how scheduler, IFLP-predictor, `pyNVML` actuation, and GPU busy time compose the total wall-clock time. No Overlap executes all CPU-side stages synchronously before GPU forward progress, exposing 27.7% of total time as GPU stall. Overlap pipelines next-iteration scheduling and prediction with current-iteration GPU execution and decouples DVFS actuation into a separate process, reducing the GPU stall to 3.6%.

In contrast, the Overlap design pipelines next-iteration scheduling and prediction with current-iteration GPU execution while decoupling DVFS actuation from the inference process. This hides most of the scheduler, IFLP, and `pyNVML` overhead from the GPU critical path. The GPU-visible stall is reduced to only 0.65 s, or 3.6% of the total 17.95 s execution time, while the GPU busy time is 17.30 s (96.4%). Correspondingly, the total execution time decreases from 24.62 s to 17.95 s, yielding a 6.67 s reduction. Overall, the runtime layer cuts the exposed CPU-side control overhead from 27.7% to 3.6% of total execution time, bringing the residual overhead introduced by EcoInfer to a level acceptable for latency-sensitive serving.

These results demonstrate that runtime optimization is necessary to make iteration-level DVFS practical in latency-sensitive serving. Without overlap, the scheduling, prediction, and `pyNVML` actuation costs are all exposed on the GPU-critical path, causing substantial delay to GPU forward progress even when the controller itself is effective. By contrast, asynchronous pipelining and decoupled DVFS actuation move most of this control cost off the GPU-visible critical path, substantially reducing exposed stall time and improving effective GPU utilization.

5.7. Summary

Overall, the evaluation supports the central claim of this paper: iteration-level GPU DVFS can improve the energy efficiency of LLM serving without materially degrading service quality provided that control decisions are both accurate and efficiently integrated into the runtime. EcoInfer achieves this combination in practice. Across diverse models and request rates, it reduces energy consumption while maintaining SLO attainment. Its frequency trace confirms workload-responsive adaptation, and its ablation results show that both predictor quality and runtime overhead management are necessary to realize the end-to-end benefit. These findings indicate that fine-grained DVFS is not only a promising optimization opportunity for LLM serving but also a deployable systems mechanism when paired with appropriate prediction and runtime support.

6. Discussion

Our results show that the benefit of fine-grained DVFS in LLM serving does not come from uniformly lowering GPU frequency but from converting iteration-level latency slack into controlled frequency reductions. The gain is most pronounced when serving iterations are decode-dominated or otherwise memory-bound, where reducing core frequency causes only modest latency inflation while still lowering power consumption substantially. This interpretation is consistent with our motivating observations: the energy–frequency relationship has a workload-dependent sweet spot, prefill and decode exhibit sharply different frequency sensitivities, and ongoing model and hardware trends are pushing inference further toward the memory-bound regime.

From a systems perspective, the results also suggest that control accuracy alone is insufficient. A frequency controller can be theoretically sound yet still fail to deliver end-to-end benefit if its online overhead remains exposed on the serving critical path. Our ablation results therefore highlight an equally important point: practical iteration-level DVFS requires co-design between decision quality and runtime integration. In EcoInfer, the predictor provides sufficiently accurate latency estimates for online control, while the decision cache and asynchronous runtime support prevent prediction and DVFS actuation from eroding the energy benefit. This combination distinguishes EcoInfer from prior work that either remains at the characterization level or applies coarser-grained control before request execution.

At the same time, EcoInfer has several limitations.

Dependence on accurate latency prediction.

The controller's safety margin is fundamentally bounded by IFLP accuracy, so sustained tail-side underestimation could yield overly aggressive frequency choices. We mitigate this with two structural safeguards—the sweet-spot-informed lower bound f_{floor} and the fallback-to-$f_{\max}$ rule when no candidate satisfies B_{iter} or the waiting queue is non-empty—but a substantial workload shift would still warrant predictor recalibration.

Stability under highly bursty workloads.

IFLP is trained on steady-state samples, so sharp arrival bursts can drive batch compositions outside its training distribution. The load-aware safeguard prevents catastrophic SLO degradation by reverting to $f_{\max}$ but trades energy for latency headroom. Reported savings should therefore be read as steady-state numbers.

Hardware generality and multi-node scalability.

Our evaluation is on a single-node A100 platform. The controller and runtime overlap layer are hardware-agnostic, while IFLP and $\mathcal{F}$ depend on per-device frequency–latency/power curves and require re-profiling on newer GPUs (e.g., H100) or heterogeneous accelerators. Beyond a single node, each engine instance can run an independent iteration-level controller, making EcoInfer naturally compatible with prefill–decode disaggregation [4,28,46] and heterogeneous attention–FFN placement [29,47,48]. Quantitative cross-instance coordination is left to future work.

Single-knob optimization.

EcoInfer currently controls only GPU core frequency, whereas jointly optimizing additional knobs such as model parallelism, instance scaling, or memory-frequency policies could unlock further energy gains.

These limitations point to several promising directions for future work. One direction is to extend EcoInfer beyond the single-node setting and study how iteration-level DVFS interacts with distributed serving architectures such as prefill–decode

disaggregation [4,28,46] and heterogeneous attention–FFN placement [29,47,48]. Another is to improve portability through online adaptation, allowing the predictor and controller to recalibrate automatically under workload drift or hardware changes. More broadly, EcoInfer suggests that energy management in LLM serving should be treated as a first-class systems problem, to be co-optimized with scheduling, memory management, and model-efficiency techniques rather than applied as an after-the-fact hardware control heuristic.

7. Conclusions

In this paper, we presented EcoInfer, an iteration-level DVFS framework for improving the energy efficiency of LLM serving under latency guarantees. EcoInfer combines a lightweight iteration-level frequency–latency predictor, an SLO-aware controller that searches within a sweet-spot-guided candidate range, and a low-overhead runtime that uses decision caching and asynchronous DVFS actuation to make fine-grained control practical. Implemented on top of vLLM, EcoInfer reduces energy consumption by up to 25.4% (21.5% on average) and improves energy efficiency by 1.28× in TPJ while maintaining a nearly unchanged SLO attainment rate. These results show that fine-grained GPU frequency control is not merely a theoretical energy-saving opportunity for LLM serving. With accurate prediction and careful runtime integration, it can be realized as a practical systems mechanism for latency-sensitive deployments.

Author Contributions: Conceptualization, Q.H. and J.L.; methodology, Q.H.; software, Q.H.; validation, Q.H. and J.L.; formal analysis, Q.H.; investigation, Q.H.; resources, J.L.; data curation, Q.H.; writing—original draft preparation, Q.H.; writing—review and editing, Q.H. and J.L.; visualization, Q.H.; supervision, J.L.; project administration, J.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Natural Science Foundation of China grant number 62232012.

Data Availability Statement: The datasets used in this study are publicly available. The ShareGPT dataset is available at https://modelscope.cn/datasets/gliang1001/ShareGPT_V3_unfiltered_cleaned_split (accessed on 1 April 2026), and the GSM8K dataset is available at https://modelscope.cn/datasets/modelscope/gsm8k (accessed on 1 April 2026). The source code for EcoInfer will be released at https://github.com/GARRYHU/EcoInfer-Code (accessed on 3 May 2026).

Acknowledgments: The authors would like to thank Mingzhen Han and other lab colleagues for helpful discussions and general support during the course of this work. During the preparation of this manuscript, the authors used ChatGPT 5.5 for language polishing. The authors reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Samsi, S.; Zhao, D.; McDonald, J.; Li, B.; Michaleas, A.; Jones, M.; Bergeron, W.; Kepner, J.; Tiwari, D.; Gadepally, V. From words to watts: Benchmarking the energy costs of large language model inference. In *Proceedings of the 2023 IEEE High Performance Extreme Computing Conference (HPEC)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–9.
2. Stojkovic, J.; Choukse, E.; Zhang, C.; Goiri, I.; Torrellas, J. Towards greener LLMs: Bringing energy-efficiency to the forefront of LLM inference. *arXiv* **2024**, arXiv:2403.20306. [CrossRef]
3. Agrawal, A.; Kedia, N.; Panwar, A.; Mohan, J.; Kwatra, N.; Gulavani, B.; Tumanov, A.; Ramjee, R. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), Santa Clara, CA, USA, 10–12 July 2024; pp. 117–134.
4. Zhong, Y.; Liu, S.; Chen, J.; Hu, J.; Zhu, Y.; Liu, X.; Jin, X.; Zhang, H. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), Santa Clara, CA, USA, 10–12 July 2024; pp. 193–210.
5. Agrawal, A.; Agarwal, A.; Kedia, N.; Mohan, J.; Kundu, S.; Kwatra, N.; Ramjee, R.; Tumanov, A. Etalon: Holistic performance evaluation framework for LLM inference systems. *arXiv* **2024**, arXiv:2407.07000. [CrossRef]

6. Liu, J.; Chung, J.W.; Wu, Z.; Lai, F.; Lee, M.; Chowdhury, M. Andes: Defining and enhancing quality-of-experience in LLM-based text streaming services. *arXiv* **2024**, arXiv:2404.16283.
7. Mei, X.; Wang, Q.; Chu, X. A survey and measurement study of GPU DVFS on energy conservation. *Digit. Commun. Netw.* **2017**, *3*, 89–100. [CrossRef]
8. Hisaharo, S.; Nishimura, Y.; Takahashi, A. Optimizing LLM inference clusters for enhanced performance and energy efficiency. *TechRxiv* **2024**. [CrossRef]
9. Wilkins, G.; Keshav, S.; Mortier, R. Offline energy-optimal LLM serving: Workload-based energy models for LLM inference on heterogeneous systems. *ACM SIGEnergy Energy Inform. Rev.* **2024**, *4*, 113–119. [CrossRef]
10. Patel, P.; Choukse, E.; Zhang, C.; Goiri, I.; Warrier, B.; Mahalingam, N.; Bianchini, R. Characterizing Power Management Opportunities for LLMs in the Cloud. In *ASPLOS '24: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*; Association for Computing Machinery: New York, NY, USA, 2024; Volume 3, pp. 207–222. [CrossRef]
11. Stojkovic, J.; Zhang, C.; Goiri, Í.; Torrellas, J.; Choukse, E. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1348–1362. [CrossRef]
12. Kakolyris, A.K.; Masouros, D.; Vavaroutsos, P.; Xydis, S.; Soudris, D. throttLL'eM: Predictive GPU Throttling for Energy Efficient LLM Inference Serving. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*; IEEE: Piscataway, NJ, USA, 2025; pp. 1363–1378. [CrossRef]
13. Yuan, Z.; Shang, Y.; Zhou, Y.; Dong, Z.; Zhou, Z.; Xue, C.; Wu, B.; Li, Z.; Gu, Q.; Lee, Y.J.; et al. Llm inference unveiled: Survey and roofline model insights. *arXiv* **2024**, arXiv:2402.16363. [CrossRef]
14. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems 30*; Curran Associates, Inc.: Red Hook, NY, USA, 2017.
15. Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C.H.; Gonzalez, J.; Zhang, H.; Stoica, I. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th Symposium on Operating Systems Principles, Koblenz, Germany, 23–26 October 2023; pp. 611–626.
16. Kamath, A.K.; Prabhu, R.; Mohan, J.; Peter, S.; Ramjee, R.; Panwar, A. Pod-attention: Unlocking full prefill-decode overlap for faster LLM inference. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Rotterdam, The Netherlands, 30 March–3 April 2025; Volume 2, pp. 897–912.
17. Wang, Q.; Chu, X. GPGPU performance estimation with core and memory frequency scaling. *IEEE Trans. Parallel Distrib. Syst.* **2020**, *31*, 2865–2881. [CrossRef]
18. Fan, K.; Cosenza, B.; Juurlink, B. Accurate energy and performance prediction for frequency-scaled GPU kernels. *Computation* **2020**, *8*, 37. [CrossRef]
19. Guerreiro, J.; Ilic, A.; Roma, N.; Tomás, P. Modeling and Decoupling the GPU Power Consumption for Cross-Domain DVFS. *IEEE Trans. Parallel Distrib. Syst.* **2019**, *30*, 2494–2506. [CrossRef]
20. Afzal, A.; Hager, G.; Wellein, G. Wattlytics: A Web Platform for Co-Optimizing Performance, Energy, and TCO in HPC Clusters. *arXiv* **2026**, arXiv:2604.08182. [CrossRef]
21. Radman, G.; Alhussain, A.; Min-Allah, N. Impact of Dynamic Voltage on GPU Energy Consumption for Real-Time Systems. *Res. Sq.* **2026**. [CrossRef]
22. NVIDIA. *NVIDIA Management Library (NVML)*; NVIDIA Developer: Santa Clara, CA, USA, 2026. Available online: https://developer.nvidia.com/management-library-nvml (accessed on 1 April 2026).
23. Velicka, D.; Vysocky, O.; Yasal, O.; Riha, L. An In-Depth Study of GPU Frequency-Scaling Latency and Its Optimization on Modern Architectures. *Future Gener. Comput. Syst.* **2025**, *179*, 108331. [CrossRef]
24. Yu, G.I.; Jeong, J.S.; Kim, G.W.; Kim, S.; Chun, B.G. Orca: A Distributed Serving System for Transformer-Based Generative Models. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), Carlsbad, CA, USA, 11–13 July 2022; pp. 521–538.
25. Sun, B.; Huang, Z.; Zhao, H.; Xiao, W.; Zhang, X.; Li, Y.; Lin, W. Llumnix: Dynamic scheduling for large language model serving. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), Santa Clara, CA, USA, 10–12 July 2024; pp. 173–191.
26. Qin, R.; Li, Z.; He, W.; Cui, J.; Ren, F.; Zhang, M.; Wu, Y.; Zheng, W.; Xu, X. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. In Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST 25), Santa Clara, CA, USA, 25–27 February 2025; pp. 155–170.
27. Dao, T.; Fu, D.; Ermon, S.; Rudra, A.; Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems 35*; Curran Associates, Inc.: Red Hook, NY, USA, 2022; pp. 16344–16359.

28. Patel, P.; Choukse, E.; Zhang, C.; Shah, A.; Goiri, Í.; Maleki, S.; Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*; IEEE: Piscataway, NJ, USA, 2024; pp. 118–132.
29. Zhu, R.; Jiang, Z.; Jin, C.; Wu, P.; Stuardo, C.A.; Wang, D.; Zhang, X.; Zhou, H.; Wei, H.; Cheng, Y.; et al. Megascale-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism. In Proceedings of the ACM SIGCOMM 2025 Conference, Coimbra, Portugal, 8–11 September 2025; pp. 592–608.
30. Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. Qwen3 technical report. *arXiv* **2025**, arXiv:2505.09388. [CrossRef]
31. ShareGPT. Available online: https://sharegpt.com (accessed on 1 April 2026).
32. Rajbhandari, S.; Li, C.; Yao, Z.; Zhang, M.; Aminabadi, R.Y.; Awan, A.A.; Rasley, J.; He, Y. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In Proceedings of the International Conference on Machine Learning, Baltimore, MD, USA, 17–23 July 2022; pp. 18332–18346.
33. Dai, D.; Deng, C.; Zhao, C.; Xu, R.; Gao, H.; Chen, D.; Li, J.; Zeng, W.; Yu, X.; Wu, Y.; et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024; pp. 1280–1297.
34. Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; Ruan, C.; et al. Deepseek-v3 technical report. *arXiv* **2024**, arXiv:2412.19437.
35. Jaech, A.; Kalai, A.; Lerer, A.; Richardson, A.; El-Kishky, A.; Low, A.; Helyar, A.; Madry, A.; Beutel, A.; Carney, A.; et al. Openai o1 system card. *arXiv* **2024**, arXiv:2412.16720.
36. DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv* **2025**, arXiv:2501.12948 .
37. Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. Training Verifiers to Solve Math Word Problems. *arXiv* **2021**, arXiv:2110.14168. [CrossRef]
38. Ruan, C.; Chen, Y.; Tian, D.; Shi, Y.; Wu, Y.; Li, J.; Li, C. DynaServe: Unified and Elastic Execution for Dynamic Disaggregated LLM Serving. *arXiv* **2025**, arXiv:2504.09285. [CrossRef]
39. Agrawal, A.; Kedia, N.; Mohan, J.; Panwar, A.; Kwatra, N.; Gulavani, B.S.; Ramjee, R.; Tumanov, A. Vidur: A Large-Scale Simulation Framework for LLM Inference. In Proceedings of the 7th Conference on Machine Learning and Systems (MLSys), Santa Clara, CA, USA, 13–16 May 2024.
40. Hoerl, A.E.; Kennard, R.W. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics* **1970**, *12*, 55–67. [CrossRef]
41. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
42. Rumelhart, D.E.; Hinton, G.E.; Williams, R.J. Learning representations by back-propagating errors. *Nature* **1986**, *323*, 533–536. [CrossRef]
43. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In *KDD '16: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 785–794. [CrossRef]
44. Shoeybi, M.; Patwary, M.; Puri, R.; LeGresley, P.; Casper, J.; Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv* **2019**, arXiv:1909.08053.
45. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*; Curran Associates, Inc.: Red Hook, NY, USA, 2017.
46. Hu, C.; Huang, H.; Xu, L.; Chen, X.; Xu, J.; Chen, S.; Feng, H.; Wang, C.; Wang, S.; Bao, Y.; et al. Inference without interference: Disaggregate LLM inference for mixed downstream workloads. *arXiv* **2024**, arXiv:2401.11181. [CrossRef]
47. Wang, B.; Wang, B.; Wan, C.; Huang, G.; Hu, H.; Jia, H.; Nie, H.; Li, M.; Chen, N.; Chen, S.; et al. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *arXiv* **2025**, arXiv:2507.19427.
48. Liu, G.; Li, H.; Guo, Y.; Lyu, Y.; Zhou, M.; Liu, Y.; Li, Z.; Wang, Y. Revealing the Challenges of Attention-FFN Disaggregation for Modern MoE Models and Hardware Systems. *arXiv* **2026**, arXiv:2602.09721.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.